Document number: P1359R0
Date: 2019-02-21
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2017
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "tentatively ready" Issues for the February, 2019 (Kona) meeting

References in this document reflect the section and paragraph numbering of document WG21 N4778.

---

581. Can a templated constructor be explicitly instantiated or specialized?

Section: 12.9.1 [temp.arg.explicit] **Status:** tentatively ready **Submitter:** Mark Mitchell **Date:** 19 May 2006

Although it is not possible to specify a constructor's template arguments in a constructor invocation (because the constructor has no name but is invoked by use of the constructor's class's name), it is possible to "name" the constructor in declarative contexts: per 6.4.3.1 [class.qual] paragraph 2,

> In a lookup in which the constructor is an acceptable lookup result, if the *nested-name-specifier* nominates a class `C`, and the name specified after the *nested-name-specifier*, when looked up in `C`, is the injected-class-name of `C` (clause 10 [class]), the name is instead considered to name the constructor of class `C`... Such a constructor name shall be used only in the *declarator-id* of a declaration that names a constructor.

Should it therefore be possible to specify *template-argument*s for a templated constructor in an explicit instantiation or specialization? For example,

```
template <int dim> struct T {};
struct X {
  template <int dim> X (T<dim> &) {};
};

template X::X<> (T<2> &);
```

If so, that should be clarified in the text. In particular, 10.3.4 [class.ctor] paragraph 1 says,

> Constructors do not have names. A special declarator syntax using an optional sequence of *function-specifier*s (9.1.2 [dcl.fct.spec]) followed by the constructor's class name followed by a parameter list is used to declare or define the constructor.

This certainly sounds as if the parameter list must immediately follow the class name, with no allowance for a template argument list.

It would be worthwhile in any event to revise this wording to utilize the "considered to name" approach of 6.4.3.1 [class.qual]; as it stands, this wording sounds as if the following would be acceptable:

```
struct S {
    S();
};
S() { }    // qualified-id not required?
```

Notes from the October, 2006 meeting:

It was observed that explicitly specifying the template arguments in a constructor declaration is never actually necessary because the arguments are, by definition, all deducible and can thus be omitted.

Additional notes, October, 2018:

The wording in 12.9.1 [temp.arg.explicit] paragraph 1 refers to a “function name,” which constructors do not have, and so presumably the current wording does not permit an explicit specialization of a constructor template. Nevertheless, there is implementation divergence in the treatment of an example like:

```
class C {
  template <typename T>
  C(const T &) {}
};
template C::C<double>(const double &);
```

with some accepting and some rejecting.

Notes from the October, 2018 teleconference:

The consensus was to allow template arguments on the constructor name but not something like `C<int>::C<float>::f`.

Proposed resolution (February, 2019):

1. Cbange 12.9.1 [temp.arg.explicit] paragraph 1 as follows:

   > Template arguments can be specified when referring to a function template specialization **that is not a specialization of a constructor template** by qualifying the function template name with the list of *template-argument*s in the same way as *template-argument*s are specified in uses of a class template specialization. [*Example:*

2. Add the following as a new paragraph following 12.9.1 [temp.arg.explicit] paragraph 1:

   > **Template arguments shall not be specified when referring to a specialization of a constructor template (10.3.4 [class.ctor], 6.4.3.1 [class.qual]).**

1937. Incomplete specification of function pointer from lambda

Section: 7.5.5 [expr.prim.lambda] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-06-05

According to 7.5.5 [expr.prim.lambda] paragraph 6,

> The closure type for a non-generic *lambda-expression* with no *lambda-capture* has a public non-virtual non-explicit const conversion function to pointer to function with C ++ language linkage (9.10 [dcl.link]) having the same parameter and return types as the closure type's function call operator. The value returned by this conversion function shall be the address of a function that, when invoked, has the same effect as invoking the closure type's function call operator.

This does not mention the object for which the function call operator would be invoked (although since there is no capture, presumably the function call operator makes no use of the object pointer). This could be addressed by relating the behavior of the function call operator to a notional temporary, or the function call operator for such closure classes could be made static.

Proposed resolution (January, 2019):

1. Change 7.5.5.1 [expr.prim.lambda.closure] paragraph 7 as follows, splitting it into two paragraphs:

   > ...The value returned by this conversion function is the address of a function `F` that, when invoked, has the same effect as invoking the closure type's function call operator **on a default-constructed instance of the closure type**. `F` is a constexpr function if the function call operator is a constexpr function.
   >
   > For a generic lambda with no *lambda-capture*, the closure type has...

2. Change 7.5.5.1 [expr.prim.lambda.closure] paragraph 9 as follows:

> The value returned by any given specialization of this conversion function template is the address of a function `F` that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization **on a default-constructed instance of the closure type**. `F` is a constexpr function if the corresponding specialization is a constexpr function **and is an immediate function if the function call operator template specialization is an immediate function**. [*Note:* This will result...

1938. Should hosted/freestanding be implementation-defined?

Section: 4.1 [intro.compliance] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-06-09

Whether an implementation is hosted or freestanding is only required to be documented by the value of the `__STDC_HOSTED__` macro (14.8 [cpp.predefined]). Should this characteristic be classified as implementation-defined, thus requiring documentation?

Proposed resolution (January, 2019):

Change 15.5.1.3 [compliance] paragraph 1 as follows:

> Two kinds of implementations are defined: *hosted* and *freestanding* (4.1 [intro.compliance]) **the kind of the implementation is implementation-defined**. For a hosted implementation, this document describes the set of available headers.

2020. Inadequate description of odr-use of implicitly-invoked functions

Section: 6.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-10-08

According to 6.2 [basic.def.odr] paragraph 3,

> A function whose name appears as a potentially-evaluated expression is odr-used if it is the unique lookup result or the selected member of a set of overloaded functions (6.4 [basic.lookup], 11.3 [over.match], 11.4 [over.over]), unless it is a pure virtual function and its name is not explicitly qualified. [*Note:* This covers calls to named functions (7.6.1.2 [expr.call]), operator overloading (Clause 11 [over]), user-defined conversions (10.3.7.2 [class.conv.fct]), allocation function for placement new (7.6.2.4 [expr.new]), as well as non-default initialization (9.3 [dcl.init]). A constructor selected to copy or move an object of class type is odr-used even if the call is actually elided by the implementation (_N4750_.15.8 [class.copy]). —*end note*] An allocation or deallocation function for a class is odr-used by a new expression appearing in a potentially-evaluated expression as specified in 7.6.2.4 [expr.new] and 10.11 [class.free]. A deallocation function for a class is odr-used by a delete expression appearing in a potentially-evaluated expression as specified in 7.6.2.5 [expr.delete] and 10.11 [class.free].

There are a couple of problems with this specification. First, contrary to the note, the names of overloaded operators, conversion functions, etc., do not appear in potentially-evaluated expressions, so the normative text does not make the note true. Also, the “as specified in” references do not cover odr-use explicitly, only the invocation of the functions.

One possible way of addressing these deficiencies would be a blanket rule like,

> A function is odr-used if it is invoked by a potentially-evaluated expression.

(The existing wording about appearing in a potentially-evaluated expression would still be needed for non-call references.)

Proposed resolution (January, 2019):

1. Change 6.2 [basic.def.odr] paragraph 2 as follows:

> An expression **or conversion** is *potentially evaluated* unless it is an unevaluated operand (7.2 [expr.prop])**,** ~~or~~ a subexpression thereof**, or a conversion in an initialization or conversion sequence in such a context**. The set of *potential results* of an expression e is defined as follows:...

2. Cbange 6.2 [basic.def.odr] paragraph 3 as follows:

> A function is *named by an expression* as follows:
>
> - A function ~~whose name appears in an expression~~ is named by ~~that~~ **an** expression **or conversion** if it is the unique ~~lookup~~ result **of a name lookup** or the selected member of a set of overloaded functions (6.4 [basic.lookup], 11.3 [over.match], 11.4 [over.over]) **in an overload resolution performed as part of forming that expression or conversion**, unless it is a pure virtual function and either ~~its name is not~~ **the expression is not an *id-expression* naming the function with an**explicitly qualified **name** or the expression forms a pointer to member (7.6.2.1 [expr.unary.op]). [*Note:* This covers taking the address of functions (7.3.3 [conv.func], 7.6.2.1 [expr.unary.op]), calls to named functions (7.6.1.2 [expr.call]), operator overloading (Clause 11 [over]), user-defined conversions (10.3.7.2 [class.conv.fct]), allocation functions for ~~placement~~ *new-expression*s (7.6.2.4 [expr.new]), as well as non-default initialization (9.3 [dcl.init]). A constructor selected to copy or move an object of class type is considered to be named by an expression **or conversion** even if the call is actually elided by the implementation (10.9.5 [class.copy.elision]). —*end note*]
>
> - A~~n allocation or~~ deallocation function for a class is named by a *new-expression* **if it is the single matching deallocation function for the allocation function selected by o verload resolution,** as specified in 7.6.2.4 [expr.new] and 10.11 [class.free].
>
> - A deallocation function for a class is named by a ~~delete expression~~ ***delete-expression* if it is the selected usual deallocation function** as specified in 7.6.2.5 [expr.delete] and 10.11 [class.free].

3. Change 6.2 [basic.def.odr] paragraph 7 as follows:

> A virtual member function is odr-used if it is not pure. A function is odr-used if it is named by a potentially-evaluated expression **or conversion**. A non-placement...

2051. Simplifying alias rules

Section: 7.2.1 [basic.lval] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-12-03

The aliasing rules of 7.2.1 [basic.lval] paragraph 10 were adapted from C with additions for C++. However, a number of the points either do not apply or are subsumed by other points. For example, the provision for aggregate and union types is needed in C for struct assignment, which in C++ is done via constructors and assignment operators in C++, not by accessing the complete object.

Suggested resolution:

Replace 7.2.1 [basic.lval] paragraph 10 as follows:

If a program attempts to access the stored value of an object through a glvalue whose type is not similar (7.3.5 [conv.qual]) to one of the following types the behavior is undefined: [*Footnote:... —end footnote*]

- the dynamic type of the object,
- the signed or unsigned type corresponding to the dynamic type of the object, or
- a `char` or `unsigned char` type.

Additional note, October, 2015:

It has been suggested that the aliasing rules should be extended to permit an object of an enumeration with a fixed underlying type to alias an object with that underlying type.

Proposed resolution (January, 2019):

1. Change 7.2.1 [basic.lval] paragraph 11 as follows:

If a program attempts to access the stored value of an object through a glvalue ~~of other than~~ **whose type is not similar (7.3.5 [conv.qual]) to** one of the following types the behavior is undefined:[58]

- the dynamic type of the object,
- ~~a cv-qualified version of the dynamic type of the object,~~
- ~~a type similar (as defined in 7.3.5 [conv.qual]) to the dynamic type of the object,~~
- a type that is the signed or unsigned type corresponding to the dynamic type of the object,
- ~~a type that is the signed or unsigned type corresponding to a cv-qualified version of the dynamic type of the object,~~
- ~~an aggregate or union type that includes one of the aforementioned types among its elements or non-static data members (including, recursively, an element or non-static data member of a subaggregate or contained union),~~
- ~~a type that is a (possibly cv-qualified) base class type of the dynamic type of the object,~~
- a `char`, `unsigned char`, or `std::byte` type.

If a program invokes a defaulted copy/move constructor or copy/move assignment operator for a union of type `U` with a glvalue argument that does not denote an object of type *cv* `U` within its lifetime, the behavior is undefined. [*Note:* Unlike in C, C++ has no accesses of class type. —*end note*]

2. Change 7.3.5 [conv.qual] paragraph 1 as follows:

A *cv-decomposition* of a type `T` is a sequence of cv_i and P_i such that `T` is

"$cv_0\ P_0 \cdots cv_{n-1}\ P_{n-1}\ cv_n$ `U`" for n ~~>~~ ≥ 0,

where...

2083. Incorrect cases of odr-use

Section: 6.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-02-11

The resolution of issue 1741 was not intended to cause odr-use to occur in cases where it did not do so previously. However, in an example like

```
extern int globx;
int main() {
  const int &x = globx;
  struct A {
   const int *foo() {
     return &x;
   }
  } a;
  return *a.foo();
}
```

`x` satisfies the requirements for appearing in a constant expression, but applying the lvalue-to-rvalue converstion to `x` does not yield a constant expression. Similarly,

```
struct A {
  int q;
  constexpr A(int q) : q(q) { }
  constexpr A(const A &a) : q(a.q * 2) { }
};

int main(void) {
```

```
  constexpr A a(42);
  constexpr int aq = a.q;
  struct Q {
   int foo() { return a.q; }
  } q;
  return q.foo();
}
```

a satisfies the requirements for appearing in a constant expression, but applying the lvalue-to-rvalue conversion to a invokes a non-trivial function.

Proposed resolution (January, 2019):

1. Change 6.2 [basic.def.odr] bullet 2.4 as follows:

   An expression is *potentially evaluated* unless it is an unevaluated operand (7.2 [expr.prop]) or a subexpression thereof. The set of *potential results* of an expression e is defined as follows:

   - ...
   - If e is a pointer-to-member expression (7.6.4 [expr.mptr.oper]) ~~whose second operand is a constant expression~~, the set contains the potential results of the object expression.
   - ...
   - Otherwise, the set is empty.

2. Change 6.2 [basic.def.odr] paragraph 4, converting the running text into bullets, as follows:

   A variable x whose name appears as a potentially-evaluated expression e~~x~~ is odr-used by e~~x~~ unless ~~applying the lvalue-to-rvalue conversion (7.3.1 [conv.lval]) to x yields a constant expression (7.7 [expr.const]) that does not invoke a function other than a trivial special member function (10.3.3 [special]) and, if x is an object,~~

   - **x is a reference that is usable in constant expressions (7.7 [expr.const]), or**
   - **x is a variable of non-reference type that is usable in constant expressions and has no mutable subobjects, and** e~~x~~ is an element of the set of potential results of an expression ~~e, where either~~ **of non-volatile-qualified non-class type to which** the lvalue-to-rvalue conversion (7.3.1 [conv.lval]) is applied ~~to e~~, or ~~e is~~
   - **x is a variable of non-reference type, and e is an element of the set of potential results of** a discarded-value expression (7.2 [expr.prop]) **to which the lvalue-to-rvalue conversion is not applied.**

   This resolution also resolves issues 2103 and 2170.

2103. Lvalue-to-rvalue conversion is irrelevant in odr-use of a reference

Section: 6.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-03-17

Issue 1741 accidentally caused 6.2 [basic.def.odr] to indicate that an lvalue-to-rvalue conversion is necessary for the odr-use of a reference. This is incorrect; any appearance of the reference name in a potentially-evaluated expression should require the reference to be defined.

See also issue 2083.

Proposed resolution (January, 2019):

This issue is resolved by the resolution of issue 2083.

2170. Unclear definition of odr-use for arrays

Section: 6.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-09-02

The current definition of odr-use of a variable is problematic when applied to an array:

> A variable `x` whose name appears as a potentially-evaluated expression `ex` is odr-used by `ex` unless applying the lvalue-to-rvalue conversion (7.3.1 [conv.lval]) to `x` yields a constant expression (7.7 [expr.const]) that does not invoke any non-trivial functions and, if `x` is an object, `ex` is an element of the set of potential results of an expression `e`, where either the lvalue-to-rvalue conversion (7.3.1 [conv.lval]) is applied to `e`, or `e` is a discarded-value expression (Clause 7 [expr]).

Consider an example like

```
struct S {
  constexpr static const int arr[3] = { 0, 1, 2 };
};
int i = S::arr[1];  // Should not require S::arr to be defined
```

Although the “set of potential results” test correctly handles the subscripting operation (since the resolution of issue 1926), it requires applying the lvalue-to-rvalue conversion to `S::arr` itself and not just to the result of the subscripting operation. Class objects exhibit a similar problem.

Proposed resolution (January, 2019):

This issue is resolved by the resolution of issue 2083.

2257. Lifetime extension of references vs exceptions

Section: 6.6.6 [class.temporary] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2016-04-07

There is implementation divergence on the following example:

```
#include <stdio.h>
struct A {
  bool live;
  A() : live(true) {
    static int cnt;
    if (cnt++ == 1) throw 0;
  }
  ~A() {
    fprintf(stderr, "live: %d\n", live);
    live = false;
  }
};
struct AA { A &&a0, &&a1; };
void doit() {
  static AA aa = { A(), A() };
}
int main(void) {
  try {
    doit();
  }
  catch (...) {
    fprintf(stderr, "in catch\n");
    doit();
  }
}
```

Some implementations produce

```
in catch
live: 1
live: 1
live: 0
```

While others produce

```
live: 1
in catch
live: 1
live: 1
```

With regard to the reference to which the first-constructed object of type `A` is bound, at what point should its lifetime end? Perhaps it should be specified that the lifetime of the temporary is only extended if said initialization does not exit via an exception (which means that calling `::std::exit(0)` as opposed to throwing would not result in a call to `~A()`).

See also issue 1634.

Notes from the December, 2016 teleconference:

The consensus was that the temporaries should be destroyed immediately if an exception occurs. 13.2 [except.ctor] paragraph 3 should be extended to apply to static initialization, so that even if a temporary is lifetime-extended (because it has static storage duration), it will be destroyed in case of an exception.

Proposed resolution (January, 2019):

Change 13.2 [except.ctor] paragraph 3 as follows:

> If the initialization or destruction of an object other than by delegating constructor is terminated by an exception, the destructor is invoked for each of the object's direct subobjects and, for a complete object, virtual base class subobjects, whose initialization has completed (9.3 [dcl.init]) and whose destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed. **[*Note:* If such an object has a reference member that extends the lifetime of a temporary object, this ends the lifetime of the reference member, so the lifetime of the temporary object is effectively not extended.—*end note*]** The subobjects are destroyed in the reverse order of the completion of their construction. Such destruction is sequenced before entering a handler of the *function-try-block* of the constructor or destructor, if any.

2266. Has dependent type vs is type-dependent

Section: 12.7.2.1 [temp.dep.type] **Status:** tentatively ready **Submitter:** Fedor Sergeev **Date:** 2016-05-20

According to 12.7.2.1 [temp.dep.type] bullet 6.3.2, one criterion for a name being a member of an unknown specialization is if the name is an *id-expression* denoting the member in a member access expression and

> the type of the object expression is dependent and is not the current instantiation.

This should presumably say that the object expression is type-dependent and not that it has a dependent type; “has a dependent type” should be applied only to declarations, not expressions.

Proposed resolution (February, 2019):

Change 12.7.2.1 [temp.dep.type] bullet 6.3.2 as follows:

> A name is a *member of an unknown specialization* if it is
>
> - ...
> - An *id-expression* denoting the member in a class member access expression (7.6.1.5 [expr.ref]) in which either

- the type of the object expression is the current instantiation, the current instantiation has at least one dependent base class, and name lookup of the *id-expression* does not find a member of a class that is the current instantiation or a non-dependent base class thereof; or
- ~~the type of~~ the object expression is **type-**dependent and is not the current instantiation.

2289. Uniqueness of structured binding names

Section: 6.3.1 [basic.scope.declarative] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-07-20

The current wording is not clear regarding examples like the following:

```
struct A { int x; } a;
struct B {} b; template<int> int &get(const B&);
struct C {}; int c[1];
auto [A] = a; // ok?
auto [B] = b; // ok?
auto [C] = c; // ok?
```

Notes from the April, 2017 teleconference:

Structured bindings have no C compatibility implications, so the tag/nontag treatment need not apply.

Proposed resolution (February, 2019):

Change 6.3.1 [basic.scope.declarative] paragraph 4 as follows:

> Given a set of declarations in a single declarative region, each of which specifies the same unqualified name,
>
> - they shall all refer to the same entity, or all refer to functions and function templates; or
> - exactly one declaration shall declare a class name or enumeration name that is not a typedef name and the other declarations shall all refer to the same variable, non-static data member, or enumerator, or all refer to functions and function templates; in this case the class name or enumeration name is hidden (6.3.10 [basic.scope.hiding]). [*Note:* A **structured binding (9.5 [dcl.struct.bind]),** namespace name **(9.7 [basic.namespace]),** or ~~a~~ class template name **(Clause 12 [temp])** must be unique in its declarative region ~~(9.7.2 [namespace.alias], Clause 12 [temp])~~. —*end note*]

2353. Potential results of a member access expression for a static data member

Section: 6.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2017-08-18

According to 6.2 [basic.def.odr] bullet 2.3, the potential results of a member access expression are simply the object expression. This rule incorrectly handles an example like:

```
struct X {
  static const int n = 0;
};
X x = {};

int b = x.n;
```

Because `X::n` is not one of the potential results, the expression `x.n` odr-uses `X::n`, requiring it to be defined.

Notes from the April, 2018 teleconference:

CWG agreed with the suggested direction to make the member a potential result in cases like the example.

Proposed resolution (February, 2019):

Change 6.2 [basic.def.odr] paragraph 2 as follows:

> An expression is potentially evaluated unless it is an unevaluated operand (7.2 [expr.prop]) or a subexpression thereof. The set of potential results of an expression `e` is defined as follows:
>
> - ...
> - If `e` is a class member access expression (7.6.1.5 [expr.ref]) **of the form `e1 . template`$_{opt}$ `e2` naming a non-static data member**, the set contains the potential results of ~~the object expression~~ **`e1`**.
> - **if `e` is a class member access expression naming a static data member, the set contains the *id-expression* designating the data member.**
> - If `e` is a pointer-to-member expression (7.6.4 [expr.mptr.oper]) ~~whose second operand is a constant expression~~ **of the form `e1 .* e2`**, the set contains the potential results of ~~the object expression~~ **`e1`**.
> - ...

2354. Extended alignment and object representation

Section: 6.6.5 [basic.align] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2017-09-05

There is implementation variance in the treatment of an example like:

```
enum struct alignas(64) A {};
A a[10];
```

The Standard does not appear to indicate whether padding bits are added to the object representation for an enumeration or not, affecting the result of `sizeof` and the alignment of the array elements.

If the extended alignment does not affect the object representation of the type, it would be useful to specify that the array declaration is ill-formed because it requires violating the alignment requirement (and similarly for an array *new-expression* where the bound is known at compile time).

Notes from the April, 2018 teleconference:

It appears that the same question exists for class types that are directly given extended alignment, as opposed to receiving it from a subobject.

Notes from the November, 2018 meeting:

CWG agreed to remove permission for `alignas` to be applied to enumerations. The class case is already handled by the wording in 7.6.2.3 [expr.sizeof] paragraph 2 that the size of a class "includes any padding required for placing objects of that type in an array."

Proposed resolution (December, 2018):

Change 9.11.2 [dcl.align] paragraph 1 as follows:

> An *alignment-specifier* may be applied to a variable or to a class data member, but it shall not be applied to a bit-field, a function parameter, or an *exception-declaration* (13.3 [except.handle]). An *alignment-specifier* may also be applied to the declaration of a class (in an *elaborated-type-specifier* (9.1.7.3 [dcl.type.elab]) or *class-head* (Clause 10 [class]), respectively) ~~and to the declaration of an enumeration (in an *opaque-enum-declaration* or *enum-head*, respectively (9.6 [dcl.enum]))~~. An *alignment-specifier* with an ellipsis...

2365. Confusing specification for `dynamic_cast`

Section: 7.6.1.7 [expr.dynamic.cast] **Status:** tentatively ready **Submitter:** Shiyao Ma **Date:** 2017-02-08

From editorial issue 1453.

According to 7.6.1.7 [expr.dynamic.cast] paragraph 4,

> If the value of `v` is a null pointer value in the pointer case, the result is the null pointer value of type `T`.

Paragraph 5 deals with the case of a simple up-cast, where no runtime type identification is required. Paragraph 6 says,

> Otherwise, `v` shall be a pointer to or a glvalue of a polymorphic type (10.6.2 [class.virtual]).

This organization of the material makes it sound as if the requirement for polymorphic class types does not apply if the argument is a null pointer value, which, of course, cannot be determined at compile time. The intent is that a null pointer value argument produces a null pointer value result, regardless of whether the relationship between the classes requires runtime type identification or not, but that the requirement for polymorphic classes applies for all casts that are not simple up-casts. The wording should be clarified.

Proposed resolution (November, 2018):

Change 7.6.1.7 [expr.dynamic.cast] paragraphs 4-6 as follows:

> ~~If the value of `v` is a null pointer value in the pointer case, the result is the null pointer value of type `T`.~~
>
> If `T` is "pointer to *cv1* `B`" and `v` has type "pointer to *cv2* `D`" such that `B` is a base class of `D`, the result is a pointer to the unique `B` subobject of the `D` object pointed to by `v`**, or a null pointer value if `v` is a null pointer value**. Similarly, if `T` is...
>
> Otherwise, `v` shall be a pointer to or a glvalue of a polymorphic type (10.6.2 [class.virtual]).
>
> **If `v` is a null pointer value, the result is a null pointer value.**

2368. Differences in relational and three-way constant comparisons

Section: 7.7 [expr.const] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2017-11-11

According to 7.7 [expr.const] bullets 2.21 and 2.22, the characteristics of three-way and relational comparisons that disqualify them as constant expressions are different:

- a three-way comparison (7.6.8 [expr.spaceship]) comparing pointers that do not point to the same complete object or to any subobject thereof;
- a relational (7.6.9 [expr.rel]) or equality (7.6.10 [expr.eq]) operator where the result is unspecified;

These are *not* equivalent, with odd results:

```
struct A {
  int a;
private:
  int b;
  constexpr auto f() { return &a < &b; }    // not constant
  constexpr auto g() { return &a <=> &b; } // returns unspecified value
};
```

Similarly,

```
struct B { int n; };
struct C : B { int m; } c;
```

```
constexpr auto x = &c.n < &c.m;    // not constant
constexpr auto y = &c.n <=> &c.m; // returns unspecified value
```

The three-way rule seems to be the correct one, but additional wording is needed in 7.6.9 [expr.rel] to specify the relational ordering within a single object: addresses of subobjects of the same complete object should be weakly ordered, and when restricted to subobjects that are not permitted to have the same address, should be totally ordered.

Notes from the October, 2018 teleconference:

The consensus of CWG was to make the 3-way operator cases non-constant, as the relational cases are.

Proposed resolution (November, 2018):

Change 7.7 [expr.const] bullets 2.22 and 2.23 as follows, merging tbe bullets:

> An expression `e` is a *core constant expression* unless the evaluation of `e`, following the rules of the abstract machine (6.8.1 [intro.execution]), would evaluate one of the following expressions:
>
> - ...
> - a three-way comparison (7.6.8 [expr.spaceship])**,** ~~comparing pointers that do not point to the same complete object or to any subobject thereof;~~
> - ~~a~~ relational (7.6.9 [expr.rel])**,** or equality (7.6.10 [expr.eq]) operator where the result is unspecified;
> - ...

2372. Incorrect matching rules for block-scope `extern` declarations

Section: 6.5 [basic.link] **Status:** tentatively ready **Submitter:** Chen Fuxingi **Date:** 2017-12-17

According to 6.5 [basic.link] paragraph 6,

> The name of a function declared in block scope and the name of a variable declared by a block scope `extern` declaration have linkage. If there is a visible declaration of an entity with linkage having the same name and type, ignoring entities declared outside the innermost enclosing namespace scope, the block scope declaration declares that same entity and receives the linkage of the previous declaration. If there is more than one such matching entity, the program is ill-formed. Otherwise, if no matching entity is found, the block scope entity receives external linkage.

The requirement that the entities have the same type does not cover all cases that it should. Consider an example like:

```
static int a[3];
void g() {
  printf("%p\n", (void*)a);
  extern int a[];
  printf("%p\n", (void*)a);
}
```

According to the cited wording, the block-scope declaration of `a` does not match the namespace scope declaration because `int[]` and `int[3]` are different types, thus the first reference to `a` refers to the static variable while the second one refers to a variable `a` defined in some other translation unit. This is clearly not intended, and current implementations treat both references as referring to the static variable.

Notes from the October, 2018 teleconference:

CWG agreed with the direction and noted that a similar situation occurs with `extern "C"` functions.

Proposed resolution (November, 2018):

Change 6.5 [basic.link] paragraph 6 as follows:

The name of a function declared in block scope and the name of a variable declared by a block scope `extern` declaration have linkage. If there is a visible declaration of an entity with linkage ~~having the same name and type~~, ignoring entities declared outside the innermost enclosing namespace scope, **such that the block scope declaration would be a (possibly ill-formed) redeclaration if the two declarations appeared in the same declarative region,** the block scope declaration declares that same entity and receives the linkage of the previous declaration. If there is more than one such matching entity, the program is ill-formed. Otherwise, if no matching entity is found, the block scope entity receives external linkage. If, within a translation unit, the same entity is declared with both internal and external linkage, the program is ill-formed. [*Example:*

```
static void f();
extern "C" void h();
static int i = 0;       // #1
void g() {
  extern void f();      // internal linkage
  extern void h();      // C language linkage

  int i;                // #2: i has no linkage
  {
    extern void f();    // internal linkage
    extern int i;       // #3: external linkage, ill-formed
  }
}
```

Without the declaration at line #2, the declaration at line #3 would link with the declaration at line #1. Because the declaration with internal linkage is hidden, however, #3 is given external linkage, making the program ill-formed. — *end example*]

2379. Missing prohibition against `constexpr` in `friend` declaration

Section: 12.6.4 [temp.friend] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-06-14

According to 12.6.4 [temp.friend] paragraph 8,

> When a friend declaration refers to a specialization of a function template, the function parameter declarations shall not include default arguments, nor shall the inline specifier be used in such a declaration.

Presumably this should also include the `constexpr` specifier.

Notes from the December, 2018 teleconference:

This should also cover the newly-added `consteval` specifier.

Proposed resolution (February, 2019):

Change 12.6.4 [temp.friend] paragraph 8 as follows:

> When a friend declaration refers to a specialization of a function template, the function parameter declarations shall not include default arguments, nor shall the ~~inline specifier~~ **`inline`, `constexpr`, or `consteval` specifiers** be used in such a declaration.

2380. *capture-default* makes too many references odr-usable

Section: 6.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-06-14

The current rule for determining when a local entity is odr-usable because of a *capture-default* is too broad. For example:

```
void f() {
  int n;
  void g(int k = n);                  // ill-formed
  [](int k = n) {};                   // ill-formed
  [=](int k = n) {};                  // valid!
  [=](int k = [=]{ return n; }()) {}; // valid!
}
```

Proposed resolution (January, 2019):

Change 6.2 [basic.def.odr] bullet 9.2.2 and add to the example as follows:

> A local entity (Clause 6.2 [basic.def.odr]) is *odr-usable* in a declarative region (6.3.1 [basic.scope.declarative]) if:
>
> - ...
>
>   - ...
>
>   - the intervening declarative region is the function parameter scope of a *lambda-expression* that has a *simple-capture* naming the entity or has a *capture-default*, **and the block scope of the *lambda-expression* is also an intervening declarative region**.
>
> If a local entity is odr-used in a declarative region in which it is not odr-usable, the program is ill-formed. [*Example:*
>
> ```
> void f(int n) {
>   [] { n = 1; };         // error, n is not odr-usable due to intervening lambda-expression
>   struct A {
>     void f() { n = 2; } // error, n is not odr-usable due to intervening function definition scope
>   };
>   void g(int = n);       // error, n is not odr-usable due to intervening function parameter scope
>   [=](int k = n) {};     // error, n is not odr-usable due to being outside the block scope of the lambda-expression
>   [&] { [n]{ return n; }; }; // OK
> }
> ```

2381. Composite pointer type of pointers to plain and noexcept member functions

Section: 7.2.2 [expr.type] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2018-06-19

According to 7.3.13 [conv.fctptr] paragraph 1,

> A prvalue of type “pointer to member of type `noexcept` function” can be converted to a prvalue of type “pointer to member of type function”. The result designates the member function.

However, 7.2.2 [expr.type] paragraph 4 does not allow for this conversion in determining the composite pointer type of two pointers to member functions. The corresponding conversion is considered for pointers to non-member functions, but only qualification conversions are considered for pointers to members:

> The *composite pointer type* of two operands `p1` and `p2` having types `T1` and `T2`, respectively, where at least one is a pointer or pointer-to-member type or `std::nullptr_t`, is:
>
> - ...
>
> - if `T1` or `T2` is “pointer to noexcept function” and the other type is “pointer to function”, where the function types are otherwise the same, “pointer to function”;
>
> - ...
>
> - if `T1` is “pointer to member of `C1` of type *cv1* `U1`” and `T2` is “pointer to member of `C2` of type *cv2* `U2`” where `C1` is reference-related to `C2` or `C2` is reference-related to `C1` (9.3.3 [dcl.init.ref]), the cv-combined type of `T2` and `T1` or the cv-combined type of `T1` and `T2`, respectively;

Note also that the places that refer to “composite pointer type” only refer to 7.3.12 [conv.mem] for conversions involving pointers to members, omitting any reference to 7.3.13 [conv.fctptr] for pointers to member functions. This affects 7.6.9 [expr.rel], 7.6.10 [expr.eq], and 7.6.16 [expr.cond].

Proposed resolution (February, 2019):

1. Change 7.2.2 [expr.type] paragraph 4 as follows:

> The *composite pointer type* of two operands `p1` and `p2` having types `T1` and `T2`, respectively, where at least one is a pointer or pointer-to-member type or `std::nullptr_t`, is:
>
> - ...
> - **if `T1` or `T2` is “pointer to member of `C1` of type function”, the other type is “pointer to member of `C2` of type `noexcept` function”, and `C1` is reference-related to `C2` or `C2` is reference-related to `C1` (9.3.3 [dcl.init.ref]), where the function types are otherwise the same, “pointer to member of `C2` of type function” or “pointer to member of `C1` of type function”, respectively;**
> - if `T1` is “pointer to member of `C1` of type *cv1* ~~`U1`~~ **`U`**” and `T2` is “pointer to member of `C2` of type *cv2* ~~`U2`~~ **`U`**”**, for some non-function type `U`,** where `C1` is reference-related to `C2` or `C2` is reference-related to `C1` (9.3.3 [dcl.init.ref]), the cv-combined type of `T2` and `T1` or the cv-combined type of `T1` and `T2`, respectively;

2. Change 7.6.10 [expr.eq] paragraph 4 as follows:

> If at least one of the operands is a pointer to member, pointer-to-member conversions (7.3.12 [conv.mem])**, function pointer conversions (7.3.13 [conv.fctptr]),** and qualification conversions (7.3.5 [conv.qual]) are performed on both operands to bring them to their composite pointer type (7.2 [expr.prop]). Comparing...

3. Change 7.6.16 [expr.cond] bullet 7.4 as follows:

> Lvalue-to-rvalue (7.3.1 [conv.lval]), array-to-pointer (7.3.2 [conv.array]), and function-to-pointer (7.3.3 [conv.func]) standard conversions are performed on the second and third operands. After those conversions, one of the following shall hold:
>
> - ...
> - One or both of the second and third operands have pointer-to-member type; pointer to member conversions (7.3.12 [conv.mem])**, function pointer conversions (7.3.13 [conv.fctptr]),** and qualification conversions (7.3.5 [conv.qual]) are performed to bring them to their composite pointer type (7.2 [expr.prop]). The result is of the composite pointer type.
> - ...

2384. Conversion function templates and qualification conversions

Section: 12.9.2.3 [temp.deduct.conv] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2018-07-24

Issue 349 resulted in the following specification in 12.9.2.3 [temp.deduct.conv] paragraph 7:

> When the deduction process requires a qualification conversion for a pointer or pointer-to-member type as described above, the following process is used to determine the deduced template argument values: If `A` is a type
>
> $cv_{1,0}$ “pointer to...” $cv_{1,n-1}$ “pointer to” $cv_{1,n}$ `T1`
>
> and `P` is a type
>
> $cv_{2,0}$ “pointer to...” $cv_{2,n-1}$ “pointer to” $cv_{2,n}$ `T2`

then the cv-unqualified `T1` and `T2` are used as the types of `A` and `P` respectively for type deduction. [*Example:*

```
struct A {
  template <class T> operator T***();
};
A a;
const int * const * const * p1 = a; // T is deduced as int, not const int
```

—*end example*]

This rule is not widely implemented and may not be desirable. Should it be removed?

Proposed resolutions (December, 2018):

Delete 12.9.2.3 [temp.deduct.conv] paragraph 7:

~~When the deduction process requires a qualification conversion for a pointer or pointer-to-member type as described above, the following process is used to determine the deduced template argument values: If `A` is a type~~

~~$cv_{1,0}$ "pointer to..." $cv_{1,n-1}$ "pointer to" $cv_{1,n}$ `T1`~~

~~and `P` is a type~~

~~$cv_{2,0}$ "pointer to..." $cv_{2,n-1}$ "pointer to" $cv_{2,n}$ `T2`~~

~~then the cv-unqualified `T1` and `T2` are used as the types of `A` and `P` respectively for type deduction. [*Example:*~~

```
~~struct A {~~
~~  template <class T> operator T***();~~
~~};~~
~~A a;~~
~~const int * const * const * p1 = a; // T is deduced as int, not const int~~
```

~~—*end example*~~]

2385. Lookup for *conversion-function-id*s

Section: 7.5.4.2 [expr.prim.id.qual] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2018-07-31

According to 7.5.4.2 [expr.prim.id.qual] paragraph 5,

> In a *qualified-id*, if the *unqualified-id* is a *conversion-function-id*, its *conversion-type-id* shall denote the same type in both the context in which the entire *qualified-id* occurs and in the context of the class denoted by the *nested-name-specifier*.

However, 6.4.5 [basic.lookup.classref] paragraph 7 says,

> If the *id-expression* is a *conversion-function-id*, its *conversion-type-id* is first looked up in the class of the object expression and the name, if found, is used. Otherwise it is looked up in the context of the entire *postfix-expression*. In each of these lookups, only names that denote types or templates whose specializations are types are considered.

The latter was changed by issue 1111. It seems the former may have been overlooked in that change.

Proposed resolution (February, 2019):

Change 7.5.4.2 [expr.prim.id.qual] paragraph 4 as follows:

> In a *qualified-id*, if the *unqualified-id* is a *conversion-function-id*, its *conversion-type-id* ~~shall denote the same type in both the context in which the entire *qualified-id* occurs and in the context of the class denoted by the *nested-name-specifier*~~ **is first looked up in the class denoted by the *nested-name-specifier* of the *qualified-id* and the name, if found, is used. Otherwise, it is looked up in the context in which the entire *qualified-id* occurs. In**

each of these lookups, only names that denote types or templates whose specializations are types are considered.

2386. tuple_size requirements for structured binding

Section: 9.5 [dcl.struct.bind] **Status:** tentatively ready **Submitter:** Stephan T. Lavavej **Date:** 2018-09-19

According to 9.5 [dcl.struct.bind] paragraph 4,

> Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression.

However, the common idiom in the library is that SFINAE tests the presence or absence of a `value` member rather than the completeness of the class; see 19.5.3.6 [tuple.helper] paragraph 4. The core language requirement should be changed to match the common library practice.

Proposed resolution (December, 2018):

Change 9.4 [dcl.fct.def] paragraph 4 as follows:

> Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete **class** type **with a member `value`**, the expression std::tuple_size<E>::value shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression...

2387. Linkage of const-qualified variable template

Section: 6.5 [basic.link] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2018-09-28

The list in 6.5 [basic.link] paragraph 3 specifying which entities receive internal linkage does not mention variable templates, so presumably a variable template has external linkage. 12 [temp] paragraph 6 gives the impression that a specialization of a template with external linkage also has external linkage. However, current implementations appear to give internal linkage to specializations of const-qualified variable templates. Should const-qualified variable templates have internal linkage?

Notes from the December, 2018 teleconference:

CWG felt that a `const` type should not affect the linkage of a variable template or its instances.

Proposed resolution (February, 2019):

Change 6.5 [basic.link] paragraph 3 as follows:

> A name having namespace scope (6.3.6 [basic.scope.namespace]) has internal linkage if it is the name of
>
> - a variable, **variable template,** function or function template that is explicitly declared `static`; or,
> - a non-inline **non-template** variable of non-volatile const-qualified type that is neither explicitly declared `extern` nor previously declared to have external linkage; or
> - a data member of an anonymous union.
>
> **[*Note:* An instantiated variable template that has const-qualified type can have external linkage, even if not declared `extern`. —*end note*]**

2394. Const-default-constructible for members

Section: 10.3.4.1 [class.default.ctor] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-07-10

After the changes for comment RU 1 in P0490R0, a defaulted default constructor is acceptable for default initialization of a `const` object under certain circumstances; for example,

```
struct A {};
const A a;
```

is well-formed. However, default-initialization of such a class member still requires a user-provided constructor:

```
struct B { const A a; };
B b;   //error
```

Proposed resolution (November, 2018):

Change 10.3.4.1 [class.default.ctor] bullet 2.4 as follows:

> A defaulted default constructor for class `X` is defined as deleted if:
>
> - ...
> - any non-variant non-static data member of const-qualified type (or array thereof) with no *brace-or-equal-initializer* ~~does not have a user-provided default constructor~~ **is not const-default-constructible (9.3 [dcl.init])** ,
> - ...